

Practice Exercises

Exercise 1: Database Selection

Ans :-

1. Online Banking System — SQL Database

An online banking system should use an SQL database because it requires ACID properties (Atomicity, Consistency, Isolation, Durability) for secure financial transactions. Banking data such as account details, balances, and transaction history is highly structured and must always remain consistent.

2. Social Media Platform — NoSQL Database

A social media platform should use a NoSQL database because it needs to handle large volumes of unstructured or semi-structured data such as posts, comments, likes, and multimedia. NoSQL databases provide horizontal scalability and flexible schemas, which are ideal for rapidly growing user data.

3. Real-time Chat Application — NoSQL Database

A real-time chat application should use a NoSQL database since it requires fast read and write operations and must handle millions of messages in real time. NoSQL systems can scale easily and support quick data access, making them suitable for messaging applications.

4. Hospital Patient Records — SQL Database

Hospital patient records should use an SQL database because medical data is highly structured and sensitive. It requires strong consistency, reliable transactions, and well-defined relationships between tables such as patients, doctors, treatments, and appointments.

5. IoT Sensor Data Collection — NoSQL Database

IoT sensor data collection should use a NoSQL database because sensors generate massive amounts of continuous data streams. NoSQL databases can efficiently store and process large-scale time-series data and support high data ingestion rates.

Exercise 2: CAP Theorem Analysis

Ans:-

1. Stock Trading Platform — Consistency (C) and Availability (A)

A stock trading platform prioritizes Consistency and Availability. All users must see the same correct stock prices and transaction data immediately, and the system must remain available to process trades quickly. Any inconsistency could cause financial loss.

2. Content Delivery Network (CDN) — Availability (A) and Partition Tolerance (P)

A CDN prioritizes Availability and Partition Tolerance. The system must always deliver content to users even if some network nodes fail. Slight delays in synchronization are acceptable, so strict consistency is not required.

3. Airline Booking System — Consistency (C) and Partition Tolerance (P)

An airline booking system prioritizes Consistency and Partition Tolerance. It is critical that two users cannot book the same seat, so the data must always remain consistent even during network failures.

4. Video Streaming Service — Availability (A) and Partition Tolerance (P)

A video streaming service prioritizes Availability and Partition Tolerance. The service should always be accessible to users, even if some data (like view counts or recommendations) is slightly delayed or inconsistent.

Exercise 3: MongoDB Operations

Ans :-

1. Create Database and Collection
use libraryDB

```
db.createCollection("books")
```

2. Adding New Books

```
db.books.insertOne({  
  title: "Introduction to Algorithms",  
  author: "Thomas H. Cormen",  
  available: true,  
  borrowedBy: null  
})
```

```
db.books.insertOne({  
  title: "Clean Code",  
  author: "Robert C. Martin",  
  available: true,  
  borrowedBy: null  
})
```

3. Finding Books by Author

```
db.books.find({ author: "Robert C. Martin" })
```

4. Updating Book Availability

```
db.books.updateOne(  
  { title: "Clean Code" },  
  { $set: { available: false, borrowedBy: "User123" } }  
)  
  
db.books.updateOne(  
  { title: "Clean Code" },  
  { $set: { available: true, borrowedBy: null } }  
)
```

5. Tracking Borrowed Books by User

```
db.books.find({ borrowedBy: "User123" })
```

Exercise 4: JSON to BSON

Ans :-

```
{  
  orderId: NumberInt(12345),  
  orderDate: ISODate("2024-01-15T00:00:00Z"),  
  totalAmount: NumberDecimal("99.99"),  
  items: ["item1", "item2"]  
}
```

Assessment Questions

1. Key Differences Between SQL and NoSQL Databases

Ans :-

SQL Databases

- SQL databases are relational databases that store data in tables with rows and columns.
- They use structured schemas, meaning the structure of the data must be defined before storing it.
- They support ACID properties to ensure reliable transactions.
- They use Structured Query Language (SQL) for querying data.
- Examples: MySQL, PostgreSQL, Oracle, Microsoft SQL Server.

Example (SQL Table)

id	name	age
1	Rahul	21
2	Priya	22

NoSQL Databases

- NoSQL databases are non-relational and store data in flexible formats such as documents, key-value pairs, graphs, or column families.
- They have dynamic schemas, meaning different records can have different structures.

- They are designed for large-scale distributed systems and high scalability.
- Examples: MongoDB, Cassandra, Redis, CouchDB.

Example (MongoDB Document)

```
{
  "name": "Rahul",
  "age": 21,
  "skills": ["Java", "Python"]
}
```

2. CAP Theorem

Ans :-

The CAP theorem states that a distributed database system can only guarantee two out of the following three properties at the same time:

- Consistency (C): All nodes see the same data at the same time.
- Availability (A): Every request receives a response.
- Partition Tolerance (P): The system continues to operate even if network communication between nodes fails.

A distributed system cannot guarantee all three simultaneously because network partitions can occur.

When a partition happens, the system must choose between:

- Consistency: return correct data but possibly reject requests.
- Availability: respond to requests even if the data might not be fully updated.

Therefore, systems typically choose CP or AP depending on their design goals.

3. Scenarios Where MongoDB is Preferred Over a Relational Database

Ans :-

1. Social Media Applications

Social media platforms store large amounts of unstructured data like posts, comments, likes, and multimedia. MongoDB's flexible document structure allows easy storage of such data.

2. Real-time Analytics and Big Data

Applications that process large volumes of rapidly changing data benefit from MongoDB's horizontal scalability and high performance.

3. Content Management Systems (CMS)

Content systems often store documents with different structures. MongoDB allows dynamic schemas, making it easier to store articles, images, and metadata without predefined tables.

4. Why MongoDB Uses BSON Instead of JSON

Ans :-

MongoDB uses BSON (Binary JSON) internally instead of JSON for several reasons:

- **Faster Processing:** BSON is binary, so it can be parsed and processed faster by machines.
- **Additional Data Types:** BSON supports more data types such as Date, Binary, ObjectId, and Decimal128, which JSON does not support.
- **Efficient Storage:** Binary encoding improves storage efficiency and indexing performance.
- **Better Query Performance:** BSON allows MongoDB to store and retrieve documents more efficiently.

Thus, BSON improves performance, storage efficiency, and functionality compared to plain JSON.

5. MongoDB Query

Ans :-

```
db.students.find({  
  GPA: { $gt: 3.5 },  
  course: "CS101"  
})
```

MongoDB CRUD Operation

Practice Exercises

Exercise 1: Basic CRUD

```
use studentDB
db.createCollection("students")
```

1. Add New Student (Create)

```
db.students.insertOne({
  name: "Rahul Sharma",
  email: "rahul@gmail.com",
  course: "CS101",
  GPA: 3.6
})
```

Add multiple students:

```
db.students.insertMany([
  {name:"Priya Singh", email:"priya@gmail.com", course:"CS101", GPA:3.8},
  {name:"Amit Verma", email:"amit@gmail.com", course:"CS102", GPA:3.4}
])
```

2. View All Students (Read)

```
db.students.find()
```

Formatted output:

```
db.students.find().pretty()
```

3. Find Student by Email

```
db.students.findOne({ email: "priya@gmail.com" })
```

4. Update Student GPA

```
db.students.updateOne(  
  { email: "amit@gmail.com" },  
  { $set: { GPA: 3.7 } }  
)
```

5. Delete Student

Delete one student:

```
db.students.deleteOne({ email: "rahul@gmail.com" })
```

Delete multiple students:

```
db.students.deleteMany({ course: "CS102" })
```

Exercise 2: Advanced Queries

1. Find all students with GPA between 3.0 and 3.5

```
db.students.find(  
  {  
    GPA: { $gte: 3.0, $lte: 3.5 }  
  })
```

- `$gte` → greater than or equal to

- `$lte` → less than or equal to

2. Find students enrolled in more than 5 courses

(Assuming courses is an array field)

```
db.students.find({  
  courses: { $size: 5 }  
})
```

For more than 5 courses:

```
db.students.find({  
  "courses.5": { $exists: true }  
})
```

3. Get Top 10 Students by GPA

```
db.students.find()  
  .sort({ GPA: -1 })  
  .limit(10)
```

- `-1` → descending order (highest GPA first)

4. Count Students by City

```
db.students.aggregate([  
  {  
    $group: {  
      _id: "$city",  
      totalStudents: { $sum: 1 }  
    }  
  }  
])
```

Exercise 3: Schema Design

1. Course with Prerequisites

Each course can have multiple prerequisite courses, so we store them as an array.

```
{  
  _id: ObjectId("..."),  
  courseCode: "CS201",  
  courseName: "Data Structures",  
  credits: 4,  
  prerequisites: ["CS101", "CS102"]  
}
```

Explanation

- `courseCode` → unique identifier of the course
- `prerequisites` → array storing course codes required before taking this course

2. Professor with Multiple Departments

A professor may belong to more than one department, so departments are stored in an array.

```
{  
  _id: ObjectId("..."),  
  name: "Dr. Sharma",  
  email: "sharma@gla.ac.in",  
  departments: ["Computer Science", "Data Science"],  
  designation: "Associate Professor"  
}
```

Explanation

- departments → array representing multiple departments the professor works in

3. Grade with Student and Course References

Grades should reference both student and course collections using their IDs.

```
{
  _id: ObjectId("..."),
  studentId: ObjectId("64ab1234..."),
  courseId: ObjectId("64cd5678..."),
  semester: "Spring 2024",
  grade: "A"
}
```

Exercise 4: Aggregation

1. Calculate Average GPA by Department

```
db.students.aggregate([
  {
    $group: {
      _id: "$department",
      avgGPA: { $avg: "$GPA" }
    }
  }
])
```

Explanation

- \$group groups students by department.
- \$avg calculates the average GPA of students in each department.

2. Find Most Popular Courses

(Assuming students have an array field called courses)

```
db.students.aggregate([
  { $unwind: "$courses" },
  {
    $group: {
      _id: "$courses",
      totalStudents: { $sum: 1 }
    }
  },
  { $sort: { totalStudents: -1 } }
])
```

Explanation

- \$unwind separates each course from the array.
- \$group counts how many students enrolled in each course.
- \$sort orders courses by popularity.

3. Generate Student Performance Report

```
db.grades.aggregate([
  {
    $lookup: {
      from: "students",
      localField: "studentId",
      foreignField: "_id",
      as: "studentInfo"
    }
  },
  {
    $lookup: {
      from: "courses",
      localField: "courseId",
```

```

    foreignField: "_id",
    as: "courseInfo"
  }
},
{
  $project: {
    studentName: { $arrayElemAt: ["$studentInfo.name", 0] },
    courseName: { $arrayElemAt: ["$courseInfo.courseName", 0] },
    grade: 1,
    semester: 1
  }
}
])

```

Assessment Questions

1. Advantages of Using Mongoose Over the Native MongoDB Driver

Mongoose is an Object Data Modeling (ODM) library for MongoDB in Node.js. It provides several advantages:

- **Schema-based structure:** Mongoose allows defining schemas for collections, which helps maintain a structured format for documents.
- **Data validation:** Built-in validation ensures that data follows the defined schema rules before saving.
- **Middleware support:** Developers can run functions before or after certain operations like saving or updating documents.
- **Simpler query building:** Mongoose provides easier methods for querying compared to the native driver.
- **Model abstraction:** It uses models to interact with collections, making code more organized and readable.

2. Difference Between findOneAndUpdate() and updateOne()

updateOne()

- Updates the first document that matches the filter.
- Returns a write result object showing how many documents were modified.
- Does not return the updated document.

Example:

```
Student.updateOne(  
  { email: "rahul@gmail.com" },  
  { $set: { GPA: 3.8 } }  
)
```

findOneAndUpdate()

- Finds a document and updates it in one operation.
- Can return the updated document if the option { new: true } is used.

Example:

```
Student.findOneAndUpdate(  
  { email: "rahul@gmail.com" },  
  { $set: { GPA: 3.8 } },  
  { new: true }  
)
```

Key Difference:

updateOne() returns update status, while findOneAndUpdate() can return the actual updated document.

3. Purpose of Middleware in Mongoose

Middleware in Mongoose are functions that run before or after certain database operations.

They are used to:

- Perform validation or transformation before saving data.
- Execute logic automatically during operations like save, update, or delete.
- Implement features like password hashing, logging, or auditing.

Example:

```
studentSchema.pre("save", function(next) {  
  console.log("Student document is about to be saved");  
  next();  
});
```

Here, the middleware runs before saving a document.

4. Implementing Pagination in Mongoose

Pagination is used to limit the number of records returned per request.

It is implemented using `limit()` and `skip()`.

Example:

```
const page = 2;  
const limit = 10;
```

```
Student.find()  
  .skip((page - 1) * limit)  
  .limit(limit);
```

Explanation

- `limit()` → number of documents per page
- `skip()` → number of documents to skip based on page number

5. When should you use embedding vs referencing in MongoDB schema design?

In MongoDB schema design, embedding and referencing are two ways to represent relationships between data.

1. Embedding

Embedding means storing related data inside the same document as a nested object or array.

When to use embedding:

- When the related data is frequently accessed together.
- When the relationship is one-to-one or one-to-few.
- When the embedded data does not grow too large.
- When you want faster read performance because all data is in one document.

Example

```
{
  name: "Rahul Sharma",
  email: "rahul@gmail.com",
  courses: [
    { courseName: "CS101", grade: "A" },
    { courseName: "CS102", grade: "B" }
  ]
}
```

Here, course information is embedded inside the student document.

2. Referencing

Referencing means storing related data in separate collections and linking them using ObjectId references.

When to use referencing:

- When the relationship is one-to-many or many-to-many.
- When related data changes frequently.
- When the data size can become very large.
- When the same data must be shared across multiple documents.

Example

```
// Student document
{
  _id: ObjectId("101"),
  name: "Rahul Sharma"
}
```

```
// Course document
{
  _id: ObjectId("201"),
  courseName: "CS101"
}
```

```
// Grade document (reference)
{
  studentId: ObjectId("101"),
  courseId: ObjectId("201"),
  grade: "A"
}
```

